

OOPs

```
In [1]: ▶ def name(x):  
        print(x)
```

```
In [2]: ▶ name('Sourav')  
  
Sourav
```

```
In [14]: ▶ name('Prabh')  
  
Prabh
```

```
In [41]: ▶ class info:  
        def name1(x):  
            print(x)
```

```
In [42]: ▶ name1()
```

NameError

Traceback (most recent call last)

Cell In[42], line 1

----> 1 name1()

NameError: name 'name1' is not defined

```
In [44]: ▶ obj = info()  
obj
```

```
Out[44]: <__main__.info at 0x2bb1a3f9d10>
```

```
In [45]: ▶ obj.name1('Sourav')
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[45], line 1  
----> 1 obj.name1('Sourav')  
  
TypeError: info.name1() takes 1 positional argument but 2 were given
```

```
In [46]: ▶ class info:  
        def name1(self, x):  
            print(x)  
  
obj = info()
```

```
In [47]: ▶ obj.name1('Sourav')
```

```
Sourav
```

```
In [ ]: ▶
```

```
In [48]: ► class info:

    def name(self, x):
        print(x)

    def age(self, y):
        print(y)

    def city(self, z):
        print(z)

obj = info()
```

```
In [49]: ► obj.name('Rupesh')
obj.age(26)
obj.city('Noida')
```

Rupesh
26
Noida

```
In [50]: ► obj.name('Honey')
```

Honey

```
In [52]: ► obj.age(30)
```

30

```
In [ ]: ►
```

class and objects

```
In [53]: ► class Person:

    def user_info(self, name, age):

        n = name
        a = age

        return n, a

p = Person()
```

```
In [54]: ► p.user_info('Harry', 30)
```

```
Out[54]: ('Harry', 30)
```

```
In [ ]: ►
```

```
In [56]: ► class Person:

    def user_info(self, name, age):
        print(f'Name: {name}, Age: {age}')

p = Person()
p.user_info('Shubham', 23)
```

```
Name: Shubham, Age: 23
```

```
In [ ]: ►
```

```
In [ ]: ►
```

Constructor

```
In [59]: ▶ class Student:

    def __init__(self, name, age, marks):

        self.name = name
        self.age = age
        self.marks = marks

        print(f'Name: {self.name}, Age: {self.age}, Marks: {self.marks}')

obj = Student('Deepak', 25, 95)
```

Name: Deepak, Age: 25, Marks: 95

In []: ▶

```
In [61]: ▶ class Student:

    def __init__(self, name, age, marks):

        self.name = name
        self.age = age
        self.marks = marks

    def display_info(self):
        print(f'Name: {self.name}, Age: {self.age}, Marks: {self.marks}')

obj = Student('Rashmi', 21, 90)
obj.display_info()
```

Name: Rashmi, Age: 21, Marks: 90

In []: ▶

4 Pillers of OOPs -

- Inheritance
- Polymorphism
- Encapsulation
- Data Abstraction

In []: ▶

Inheritance

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

In []: ▶

Single Inheritance

In []: ▶

```
# Parent Class > Child Class  
# Base Class > Derived Class  
# Root Class > Branch Class
```

```
In [70]: ▶ # 1 parent class and 1 child class

class Teacher:

    def math(self):
        print('Math Knowledge')

class Student(Teacher):

    def learning_math(self):
        print('Learning Math from Teacher.')

T = Teacher()
T.math()
print()

S = Student()
S.learning_math()
S.math()
```

Math Knowledge

Learning Math from Teacher.
Math Knowledge

In []: ▶

Multiple Inheritance

```
In [85]: # Parent Class 1  
class Father:  
  
    def Anger(self):  
        print('Angry Mood!')  
  
# Parent Class 2  
class Mother:  
  
    def Care(self):  
        print('Caring Nature!')  
  
# Child Class  
class Child(Father, Mother):  
  
    def Qualities(self):  
        print('Acquiring the qualities of both parent classes')  
  
f = Father()  
f.Anger()  
  
m = Mother()  
m.Care()  
  
print()  
  
ch = Child()  
ch.Qualities()  
  
print()  
ch.Anger()  
ch.Care()
```


Angry Mood!
Caring Nature!

Acquiring the qualities of both parent classes

Angry Mood!
Caring Nature!

In []: ▶

Multilevel Inheritance

```
In [101]: ► class Grandfather:

    def Wise(self):
        print('Quality by Grandfather!')

class Father(Grandfather):

    def Anger(self):
        print('Quality by Father!')

class Child(Father):

    def all_qualities(self):
        print('Qualities by Generation.')

gf = Grandfather()
f = Father()
ch = Child()

gf.Wise()
f.Anger()
ch.all_qualities()

print('\n Father acquiring the property by grandfather:')
f.Wise()

print('\n Child acquiring the property by father and grandfather:')
ch.Anger()
ch.Wise()
```

```
Quality by Grandfather!  
Quality by Father!  
Qualities by Generation.
```

```
Father acquiring the property by grandfather:  
Quality by Grandfather!
```

```
Child acquiring the property by father and grandfather:  
Quality by Father!  
Quality by Grandfather!
```

In []: ▶

Hieraichical Inheritance


```
In [14]: ► class Vehical:

    def Petrol_Pump(self, vehical):
        print(f'{vehical} start with Petrol/Diesel/CNG!')

class Car(Vehical):
    def drive(self):
        print('Car Driving!')

class Bike(Vehical):
    def ride(self):
        print('Bike Riding!')

class Auto(Vehical):

    def Fly(self):
        print('Auto Flying!')

class Bus(Vehical):
    def drive(self):
        print('Bus Driving')

# vh = Vehical()
# vh.Petrol_Pump()

c = Car()
c.drive()
c.Petrol_Pump('Car')

print()
b = Bike()
b.ride()
b.Petrol_Pump('Bike')

print()
a = Auto()
a.Fly()
a.Petrol_Pump('Auto')

print()
bus = Bus()
```

```
bus.drive()  
bus.Petrol_Pump('Bus')
```

Car Driving!
Car start with Petrol/Diesel/CNG!

Bike Riding!
Bike start with Petrol/Diesel/CNG!

Auto Flying!
Auto start with Petrol/Diesel/CNG!

Bus Driving
Bus start with Petrol/Diesel/CNG!

In []: ▶

Encapsulation

```
In [29]: ▶ class Student:

    def __init__(self, name, roll_no):

        self.name = name
        self.roll_no = roll_no
        self.marks = []

    def add_marks(self, mark):
        if mark >= 0 and mark <=100:
            self.marks.append(mark)
        else:
            print('Invalid Marks, Mark should be in between 0 to 100.')

    def get_avg_marks(self):

        if len(self.marks) > 0:
            return sum(self.marks)/len(self.marks)
        else:
            return 0

st = Student('Deepak', 101)

st.add_marks(80)
st.add_marks(95)
st.add_marks(86)
st.add_marks(94)

print(f'Student Name: {st.name}')
print(f'Student NamRoll No.: {st.roll_no}')
print()
print(f'Student Marks: {st.get_avg_marks()} %')
```

Student Name: Deepak
Student NamRoll No.: 101

Student Marks: 88.75 %

```
In [30]: ▶ st = Student('Rashmi', 102)

st.add_marks(70)
st.add_marks(85)
st.add_marks(96)
st.add_marks(64)

print(f'Student Name: {st.name}')
print(f'Student NamRoll No.: {st.roll_no}')
print()
print(f'Student Marks: {st.get_avg_marks()} %')
```

Student Name: Rashmi
Student NamRoll No.: 102

Student Marks: 78.75 %

In []: ▶

In []: ▶

Polymorphism

```
In [ ]: ▶ # He -

# Father
# Son
# Brother
# Friend
# Boy
```



```
In [41]: ► class Circle:
            def __init__(self, radius):
                self.radius = radius

            def area(self):
                return 3.14 * self.radius**2

        class Rectangle:
            def __init__(self, length, width):
                self.length = length
                self.width = width

            def area(self):
                return self.length * self.width

        c = Circle(6)
        print(f'Circle Area: {c.area()} ')

        r = Rectangle(5, 4)
        print(f'Rectangle Area: {r.area()} sq.ft.')
```

```
Circle Area: 113.04
Rectangle Area: 20 sq.ft.
```

```
In [ ]: ►
```

Method Overriding

```
In [45]: ► class Dog:
            def speak(self, voice):
                return voice

            class Cat:
                def speak(self, voice):
                    return voice

            d = Dog()
            d = d.speak('Barks')
            print(f'Dog Voice: {d}')

            c = Cat()
            c = c.speak('Meow..')
            print(f'Cat Voice: {c}')
```

Dog Voice: Barks
Cat Voice: Meow..

In []: ►

Method Overloading

```
In [46]: ► class Calculator:

            def add(self, a, b):

                return a + b

            cal = Calculator()
            cal.add(4, 5)
```

Out[46]: 9

In []: ▶

In [49]: ▶

```
class Calculator:

    def add(self, a, b=6):

        return a + b

cal = Calculator()
cal.add(4)
```

Out[49]: 10

In [51]: ▶

```
class Calculator:

    def add(self, a, b=6):

        return a + b

cal = Calculator()
cal.add(4, 8)
```

Out[51]: 12

In []: ▶

```
In [55]: ► class Calculator:

    def add(self, a, b=6, c=0):

        return a + b + c

cal = Calculator()
cal.add(4)
```

Out[55]: 10

```
In [56]: ► class Calculator:

    def add(self, a, b=6, c=0):

        return a + b + c

cal = Calculator()
cal.add(4,6,8)
```

Out[56]: 18

```
In [ ]: ►
```

Data Abstraction

- Data abstraction in Python's Object-Oriented Programming (OOP) involves hiding the complex implementation details of a class from the user and only providing essential features or interfaces. Let's create a simple banking example to demonstrate data abstraction:

```
In [58]: ▶ class BankAccount:
    def __init__(self, account_number, balance):
        self._account_number = account_number
        self._balance = balance

    def deposit(self, amount):
        self._balance += amount
        print(f"Deposited {amount}. New balance is {self._balance}")

    def withdraw(self, amount):
        if amount <= self._balance:
            self._balance -= amount
            print(f"Withdraw {amount}. New balance is {self._balance}")
        else:
            print("Insufficient funds")

    def get_balance(self):
        return self._balance

# Usage
account = BankAccount("123456", 1000)
print("Initial balance:", account.get_balance())
account.deposit(500)
account.withdraw(200)
account.withdraw(1500)
```

```
Initial balance: 1000
Deposited 500. New balance is 1500
Withdraw 200. New balance is 1300
Insufficient funds
```

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

In []: ▶

